

DATA STRUCTURES AND ALGORITHMS

Unit 4

Searching and Sorting

Linear, Binary & Interpolation Search · Selection, Insertion, Bubble, Quick & Merge Sort

ROADMAP FOR THIS UNIT

What We Will Cover

01

Introduction to Searching

02

Linear Search

03

Binary Search

04

Interpolation Search

05

Introduction to Sorting

06

Selection Sort

07

Insertion Sort

08

Bubble Sort

09

Quick Sort

10

Merge Sort

11

Comparing All Algorithms

CONCEPT 01

Introduction to Searching

- Searching is the process of finding whether a particular element (the key) is present in a collection of data, and if so, locating its position.
- Every searching algorithm answers the same question — but they differ in HOW they use the structure of the data to answer it faster.
- **Two broad categories:**
 - Sequential Search — checks elements one by one (e.g. Linear Search). Works on any data, sorted or not.
 - Interval / Divide-based Search — uses the fact that data is sorted to skip large portions (e.g. Binary Search, Interpolation Search).
- **Choosing the right search algorithm depends on: is the data sorted? how large is it? how is it stored (array vs linked list)?**

Linear Search — Concept

- Also called Sequential Search — the simplest searching technique.
- Idea: start from the first element and compare it with the key, one element at a time, until either a match is found or the list ends.
- Does NOT require the data to be sorted — works on any array or list.
- Because every element may need to be checked, it is not efficient for very large datasets.
- Best suited for: small datasets, unsorted data, or data stored in a structure without random access (like a linked list).

How it moves through the array

12	7	25	3	19	8	14
0	1	2	3	4	5	6

Searching for key = 19 → checked at every index: 0,1,2,3,4 ...

12	7	25	3	19	8	14
0	1	2	3	4	5	6



checked, not a match



match found (stop here)

Linear Search — Algorithm

- **Algorithm:**
- Start at index $i = 0$
- Compare `array[i]` with the key
- If they match → return index i (search successful)
- Otherwise, increment i and repeat
- If i reaches the end of the array without a match → return -1 (not found)

```
int linearSearch(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++) {  
        if (arr[i] == key)  
            return i;  
    }  
    return -1;  
}
```

```
// Pseudocode  
LINEAR_SEARCH(A, n, key)  
    for i = 0 to n-1  
        if A[i] == key  
            return i  
    return -1
```

Linear Search — Step-by-Step Example

Array: [4, 10, 6, 15, 2, 9] Key to find = 15

4	10	6	15	2	9
---	----	---	----	---	---

$i = 0 \rightarrow \text{arr}[0] = 4 \neq 15 \rightarrow \text{move on}$

4	10	6	15	2	9
---	----	---	----	---	---

$i = 1 \rightarrow \text{arr}[1] = 10 \neq 15 \rightarrow \text{move on}$

4	10	6	15	2	9
---	----	---	----	---	---

$i = 2 \rightarrow \text{arr}[2] = 6 \neq 15 \rightarrow \text{move on}$

4	10	6	15	2	9
---	----	---	----	---	---

$i = 3 \rightarrow \text{arr}[3] = 15 = 15 \rightarrow \text{MATCH! return index 3}$

4 comparisons were needed to find the key — in the worst case, all n elements are checked.

Linear Search — Complexity Analysis

$O(1)$

Best Case — key at the first index

$O(n)$

Average Case — key somewhere in the middle

$O(n)$

Worst Case — key at the end / not present

- Space Complexity: $O(1)$ — no extra memory is used, search happens in place.
- Advantages: simple to implement; works on unsorted data; works on any data structure (array, linked list).
- Disadvantages: slow for large datasets since it may need to check every element.
- Rule of thumb: use Linear Search only when data is small or unsorted — otherwise a sorted-data search (Binary / Interpolation) is far faster.

Binary Search — Concept

- **Precondition: the array MUST be sorted before Binary Search can be applied.**
- Idea: repeatedly divide the search interval in half.
- Compare the key with the middle element:
 - If key == mid element → found!
 - If key < mid element → search only the LEFT half
 - If key > mid element → search only the RIGHT half
- Each comparison eliminates roughly half of the remaining elements — this is what makes it so fast.
- **This is a Divide and Conquer strategy applied to searching.**

Binary Search — Algorithm

- **Algorithm (iterative):**
- Set $low = 0$, $high = n - 1$
- While $low \leq high$:
 - $mid = (low + high) / 2$
 - If $arr[mid] == key \rightarrow$ return mid
 - Else if $key < arr[mid] \rightarrow high = mid - 1$
 - Else $\rightarrow low = mid + 1$
- If $low > high \rightarrow$ key not found, return -1

```
int binarySearch(int arr[], int n, int key) {  
    int low = 0, high = n - 1;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (arr[mid] == key)  
            return mid;  
        else if (key < arr[mid])  
            high = mid - 1;  
        else  
            low = mid + 1;  
    }  
    return -1;  
}
```

Binary Search — Step-by-Step Example

Sorted Array: [2, 4, 6, 8, 10, 12, 14, 16] Key to find = 12

2	4	6	8	10	12	14	16
0	1	2	3	4	5	6	7

low=0, high=7 → mid=3 → arr[3]=8 < 12 → search RIGHT half

2	4	6	8	10	12	14	16
0	1	2	3	4	5	6	7

low=4, high=7 → mid=5 → arr[5]=12 = 12 → MATCH! return index 5

Only 2 comparisons were needed for 8 elements — compare to up to 8 for Linear Search.

Binary Search — Complexity & Recursive Form

$O(1)$

Best Case — key is the middle element

$O(\log n)$

Average Case

$O(\log n)$

Worst Case — halved every step

```
int binarySearchRec(int arr[], int lo, int hi, int key) {  
    if (lo > hi) return -1;  
    int mid = (lo + hi) / 2;  
    if (arr[mid] == key) return mid;  
    if (key < arr[mid])  
        return binarySearchRec(arr, lo, mid-1, key);  
    return binarySearchRec(arr, mid+1, hi, key);  
}
```

- Space: $O(1)$ iterative, $O(\log n)$ recursive (call stack).
- For $n = 1,000,000$, Linear Search may need $\sim 1,000,000$ checks; Binary Search needs only ~ 20 .
- Trade-off: requires the array to be kept sorted.

Interpolation Search — Concept

- An improvement over Binary Search — designed for sorted AND uniformly distributed data.
- Binary Search always checks the middle element. Interpolation Search makes a smarter guess — like flipping to the right page of a dictionary based on the letter, instead of always opening to the middle.
- It estimates the probable position of the key using a formula, based on the value of the key relative to the values at the ends of the range.
- **Position formula:**

$$\text{pos} = \text{low} + \lfloor (\text{key} - \text{arr}[\text{low}]) \times (\text{high} - \text{low}) / (\text{arr}[\text{high}] - \text{arr}[\text{low}]) \rfloor$$

When data is uniformly distributed, this formula lands very close to the key's actual position — often in a single step.

Interpolation Search — Algorithm

- **Algorithm:**
- Set $low = 0$, $high = n - 1$
- While $low \leq high$ AND key lies between $arr[low]$ and $arr[high]$:
 - Compute pos using the interpolation formula
 - If $arr[pos] == key \rightarrow$ return pos
 - Else if $key < arr[pos] \rightarrow high = pos - 1$
 - Else $\rightarrow low = pos + 1$
- If the loop ends $\rightarrow$ key not found, return -1

```
int interpolationSearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high && key >= arr[low] && key <= arr[high])
    {
        int pos = low + (int)((double)(key - arr[low]) *
                               (high - low) / (arr[high] - arr[low]));
        if (arr[pos] == key) return pos;
        if (key < arr[pos]) high = pos - 1;
        else low = pos + 1;
    }
    return -1;
}
```

Interpolation Search — Step-by-Step Example

Sorted, uniform array: [10, 20, 30, 40, 50, 60, 70, 80, 90] Key to find = 70

10	20	30	40	50	60	70	80	90
0	1	2	3	4	5	6	7	8

$$\text{pos} = 0 + \lfloor (70 - 10) \times (8 - 0) / (90 - 10) \rfloor = 0 + \lfloor 480 / 80 \rfloor = 6$$

$\text{arr}[6] = 70 = \text{key} \rightarrow \text{MATCH found in a single probe — no repeated halving needed.}$

Compare: Binary Search would need 3 comparisons (mid=4→50, mid=6→70) to reach the same result; here it took one calculated probe.

SEARCH 03 · INTERPOLATION SEARCH

Interpolation Search — Complexity Analysis

$O(1)$

Best Case — formula lands exactly on the key

$O(\log \log n)$

Average Case — uniformly distributed data

$O(n)$

Worst Case — highly non-uniform data

- Space Complexity: $O(1)$.
- Requires: sorted data (like Binary Search), and works best when values are uniformly / evenly spread out.
- Worst case degrades to $O(n)$ when data is skewed or exponentially distributed — the position formula's guesses become poor.
- Real-world use: telephone directories, dictionaries, or any dataset where values are roughly evenly spaced.

RECAP

Linear vs Binary vs Interpolation Search

Feature	Linear Search	Binary Search	Interpolation Search
Precondition	None — any order	Data must be sorted	Sorted + uniformly distributed
Technique	Check one by one	Divide interval in half	Probe estimated position
Best Case	$O(1)$	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$	$O(\log \log n)$
Worst Case	$O(n)$	$O(\log n)$	$O(n)$
Space	$O(1)$	$O(1)$ / $O(\log n)$ recursive	$O(1)$

CONCEPT 05

Introduction to Sorting

- Sorting is the process of arranging elements of a list/array in a particular order — usually ascending or descending.
- **Why sort data?**
 - Makes searching much faster (enables Binary/Interpolation Search).
 - Makes data easier to analyse, display, and compare (rankings, leaderboards, reports).
- **Key properties used to classify sorting algorithms:**
 - In-place — needs only $O(1)$ extra memory, sorts within the original array.
 - Stable — equal elements keep their original relative order after sorting.
 - Comparison-based — decides order by comparing pairs of elements (all algorithms in this unit).

CONCEPT 05 · OVERVIEW

The Sorting Algorithms We Will Study

1

Selection Sort

Simple · $O(n^2)$

Repeatedly selects the smallest remaining element.

2

Insertion Sort

Simple · $O(n^2)$

Builds the sorted array one element at a time.

3

Bubble Sort

Simple · $O(n^2)$

Repeatedly swaps adjacent out-of-order elements.

4

Quick Sort

Divide & Conquer · $O(n \log n)$

Partitions around a pivot, then sorts each side.

5

Merge Sort

Divide & Conquer · $O(n \log n)$

Splits the array, sorts halves, then merges them.

Selection Sort — Concept

- Idea: divide the array into a SORTED part (left) and an UNSORTED part (right).
- In every pass, find the SMALLEST element in the unsorted part.
- Swap it with the first element of the unsorted part — growing the sorted part by one.
- Repeat until the entire array is sorted.
- The number of swaps is at most $(n - 1)$ — far fewer than Bubble Sort — but the number of comparisons is still $O(n^2)$.



Find the minimum (1) in the unsorted region, then swap it into position 0.

Selection Sort — Algorithm

- **Algorithm:**
- For $i = 0$ to $n - 2$:
 - Assume $arr[i]$ is the minimum; set $minIdx = i$
 - For $j = i+1$ to $n - 1$:
 - If $arr[j] < arr[minIdx] \rightarrow minIdx = j$
 - Swap $arr[i]$ and $arr[minIdx]$
- After the loop, the array is fully sorted.

```
void selectionSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIdx = i;  
        for (int j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIdx])  
                minIdx = j;  
        }  
        int t = arr[i];  
        arr[i] = arr[minIdx];  
        arr[minIdx] = t;  
    }  
}
```

SORT 01 • SELECTION SORT

Selection Sort — Step-by-Step Example

Array: [29, 10, 14, 37, 13]

29	10	14	37	13
----	----	----	----	----

Pass 1: min in [29,10,14,37,13] is 10 → swap with index 0

10	29	14	37	13
----	----	----	----	----

Pass 2: min in [29,14,37,13] is 13 → swap with index 1

10	13	14	37	29
----	----	----	----	----

Pass 3: min in [14,37,29] is 14 → already in place

10	13	14	37	29
----	----	----	----	----

Pass 4: min in [37,29] is 29 → swap with index 3

10	13	14	29	37
----	----	----	----	----

Sorted! [10, 13, 14, 29, 37]



sorted region



current minimum

SORT 01 · SELECTION SORT

Selection Sort — Complexity & Properties

$O(n^2)$

Best Case — still scans to find min every pass

$O(n^2)$

Average Case

$O(n^2)$

Worst Case

- Space Complexity: $O(1)$ — sorts in place.
- Stable? NOT stable in its standard form (a swap can move an equal element past another equal element).
- Number of swaps: at most $n - 1$ — the fewest of the three simple sorts, useful when writes are costly (e.g. flash memory).
- Best used when: the array is small, or minimizing the number of swaps matters more than the number of comparisons.

Insertion Sort — Concept

- Idea: build the sorted array one element at a time, exactly the way a person sorts playing cards in hand.
- Consider the first element as already 'sorted'.
- Take the next element (the key) and insert it into its correct position within the sorted part, shifting larger elements one step to the right.
- Repeat for every remaining element.
- Very efficient for data that is already almost sorted — it can finish in close to $O(n)$ time.



Key = 3 shifts left past 8 and 5 until it finds its correct sorted position.

Insertion Sort — Algorithm

- **Algorithm:**
- For $i = 1$ to $n - 1$:
 - $\text{key} = \text{arr}[i]; j = i - 1$
 - While $j \geq 0$ AND $\text{arr}[j] > \text{key}$:
 - $\text{arr}[j+1] = \text{arr}[j]; j = j - 1$
 - $\text{arr}[j+1] = \text{key}$
- After the loop, $\text{arr}[0 .. i]$ is sorted; continue to the next i .

```
void insertionSort(int arr[], int n) {  
    for (int i = 1; i < n; i++) {  
        int key = arr[i];  
        int j = i - 1;  
        while (j >= 0 && arr[j] > key) {  
            arr[j + 1] = arr[j];  
            j--;  
        }  
        arr[j + 1] = key;  
    }  
}
```

Insertion Sort — Step-by-Step Example

Array: [9, 5, 1, 4, 3]

9	5	1	4	3
---	---	---	---	---

i=1: key=5. Shift 9 right → [5, 9, 1, 4, 3]

5	9	1	4	3
---	---	---	---	---

i=2: key=1. Shift 9,5 right → [1, 5, 9, 4, 3]

1	5	9	4	3
---	---	---	---	---

i=3: key=4. Shift 9,5 right → [1, 4, 5, 9, 3]

1	4	5	9	3
---	---	---	---	---

i=4: key=3. Shift 9,5,4 right → [1, 3, 4, 5, 9]

1	3	4	5	9
---	---	---	---	---

Sorted! [1, 3, 4, 5, 9]

● sorted region (grows left to right)

SORT 02 · INSERTION SORT

Insertion Sort — Complexity & Properties

$O(n)$

Best Case — array already sorted

$O(n^2)$

Average Case

$O(n^2)$

Worst Case — array in reverse order

- Space Complexity: $O(1)$ — sorts in place.
- Stable? Yes — equal elements are never swapped past each other.
- Adaptive: performs very well on nearly-sorted data, since fewer shifts are needed.
- Widely used for small arrays, and as the final step inside faster hybrid algorithms (e.g. inside optimized Quick Sort / Merge Sort implementations).

Bubble Sort — Concept

- Idea: repeatedly step through the array, comparing each pair of ADJACENT elements, and swap them if they are in the wrong order.
- After each full pass, the largest unsorted element 'bubbles up' to its correct position at the end.
- The process repeats for $n - 1$ passes, with the sorted region growing from the right.
- An optimization: if a full pass makes NO swaps, the array is already sorted — the algorithm can stop early.



5 > 2, adjacent and out of order → swap them, then move to the next pair.

Bubble Sort — Algorithm

- **Algorithm:**
- For $i = 0$ to $n - 2$:
 - swapped = false
 - For $j = 0$ to $n - i - 2$:
 - If $arr[j] > arr[j+1] \rightarrow$ swap them; swapped = true
 - If swapped == false $\rightarrow$ break (already sorted)

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        bool swapped = false;  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int t = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = t;  
                swapped = true;  
            }  
        }  
        if (!swapped) break;    // early exit  
    }  
}
```

Bubble Sort — Step-by-Step Example (Pass 1)

Array: [5, 1, 4, 2, 8]

5	1	4	2	8
---	---	---	---	---

Compare 5,1 → $5 > 1$ → swap

1	5	4	2	8
---	---	---	---	---

Compare 5,4 → $5 > 4$ → swap

1	4	5	2	8
---	---	---	---	---

Compare 5,2 → $5 > 2$ → swap

1	4	2	5	8
---	---	---	---	---

Compare 5,8 → in order → no swap

1	4	2	5	8
---	---	---	---	---

End of Pass 1 — largest (8) is bubbled to the end

This pattern repeats for further passes, each time ignoring one more already-sorted element at the end.

SORT 03 · BUBBLE SORT

Bubble Sort — Complexity & Properties

$O(n)$

Best Case — with early-exit optimisation

$O(n^2)$

Average Case

$O(n^2)$

Worst Case — array in reverse order

- Space Complexity: $O(1)$ — sorts in place.
- Stable? Yes — equal elements are never swapped.
- Simplicity vs Performance: easiest sort to understand and trace by hand, but rarely used in production code because it makes the most swaps among the simple sorts.
- Mainly taught for its educational value in illustrating comparison-and-swap based sorting.

Quick Sort — Concept

- A Divide and Conquer algorithm — one of the fastest general-purpose sorting algorithms in practice.
- **Idea:**
 - Choose an element as the PIVOT (commonly the last, first, or a random element).
 - Partition the array so that all elements smaller than the pivot go to its left, and all larger elements go to its right.
 - The pivot is now in its final sorted position.
 - Recursively apply the same process to the left and right sub-arrays.
- Unlike Merge Sort, Quick Sort sorts in place — it does not need extra arrays for merging.

Quick Sort — Algorithm (Lomuto Partition)

- **Partition(arr, low, high):**
- pivot = arr[high]; i = low - 1
- For j = low to high - 1:
 - If $\text{arr}[j] \leq \text{pivot} \rightarrow i++$; swap arr[i], arr[j]
- Swap arr[i+1] and arr[high]; return i+1 (pivot's final index)
- **QuickSort(arr, low, high):**
- If low < high:
 - p = Partition(arr, low, high)
 - QuickSort(arr, low, p-1); QuickSort(arr, p+1, high)

```
int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i+1], a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

SORT 04 · QUICK SORT

Quick Sort — Partitioning Example

Array: [5, 2, 8, 1, 9, 3] Pivot = last element = 3

5	2	8	1	9	3
0	1	2	3	4	5

Start: scan $j=0..4$, move every element ≤ 3 to the front (tracked by pointer i)

After partitioning:

2	1	3	5	9	8
0	1	2	3	4	5

● smaller than pivot (left partition)

● pivot — now in final position (index 2)

● larger than pivot (right partition)

Quick Sort now recurses independently on [2, 1] and [5, 9, 8] — the pivot never moves again.

SORT 04 · QUICK SORT

Quick Sort — Complexity Analysis

$O(n \log n)$

Best Case — pivot always splits evenly

$O(n \log n)$

Average Case

$O(n^2)$

Worst Case — pivot is always the min/max

- Worst case happens when the array is already sorted (or reverse sorted) and the pivot is always chosen as the first/last element — partitions become very unbalanced (size 0 and $n-1$).
- This is avoided in practice using a randomly chosen pivot, or the 'median-of-three' rule.
- Space Complexity: $O(\log n)$ average (recursion stack), $O(n)$ worst case.
- Stable? No — the partitioning step can reorder equal elements.
- Generally the fastest comparison sort in practice due to good cache performance and in-place partitioning.

Merge Sort — Concept

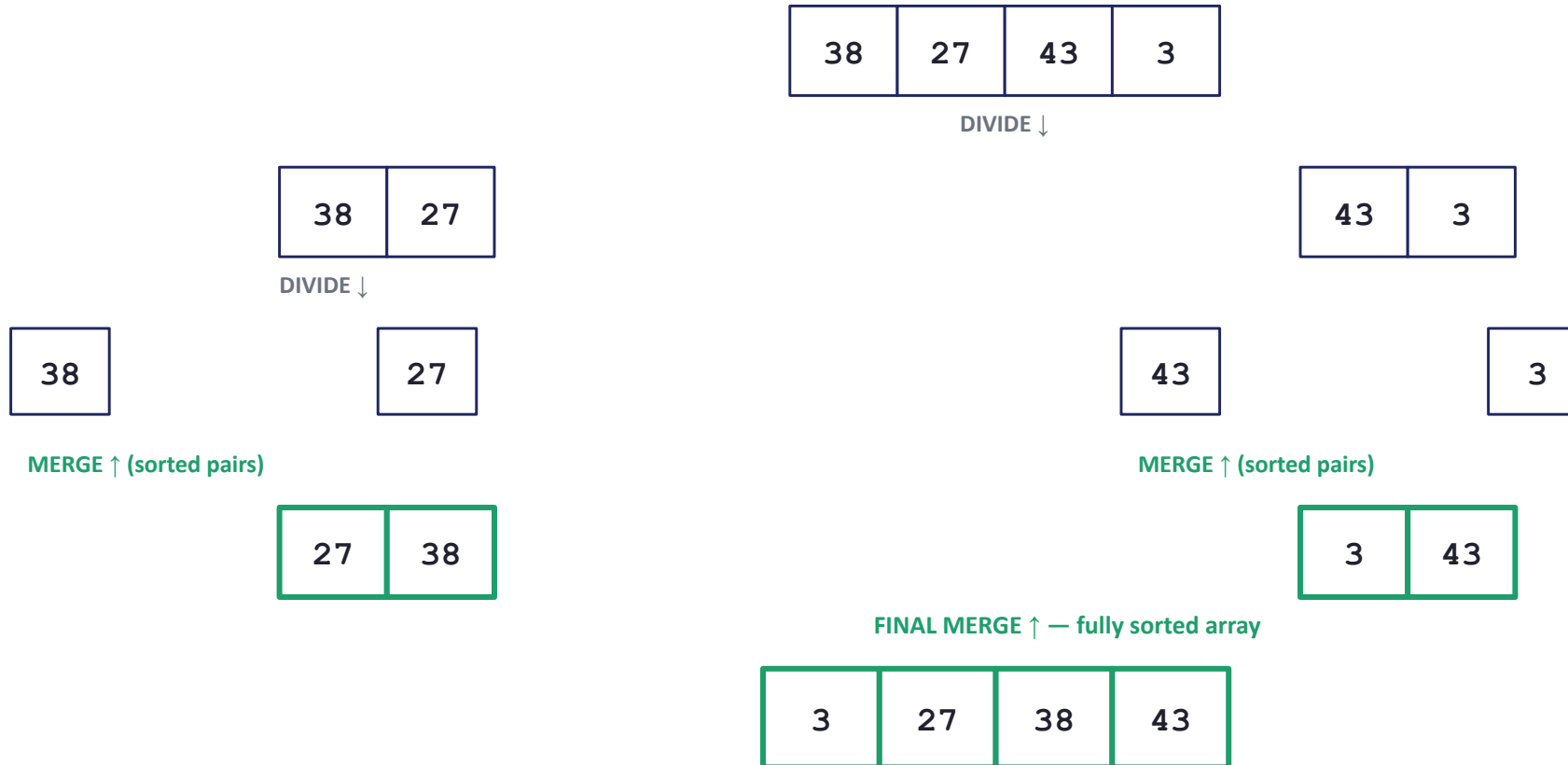
- Another Divide and Conquer algorithm, built on one key idea: merging two already-sorted arrays is easy and fast.
- **Idea:**
 - Divide the array into two halves.
 - Recursively sort each half.
 - Merge the two sorted halves back into a single sorted array.
- Base case: an array of size 0 or 1 is already sorted.
- Unlike Quick Sort, Merge Sort needs extra memory to hold the merged results — it is NOT in-place.

Merge Sort — Algorithm

- **MergeSort(arr, low, high):**
- If $low < high$:
 - $mid = (low + high) / 2$
 - MergeSort(arr, low, mid)
 - MergeSort(arr, mid+1, high)
 - Merge(arr, low, mid, high)
- **Merge(...):** copy both halves into temp arrays, then repeatedly pick the smaller front element from either temp array back into arr, until both are exhausted.

```
void mergeSort(int a[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
        mergeSort(a, low, mid);  
        mergeSort(a, mid + 1, high);  
        merge(a, low, mid, high);  
    }  
}  
  
// merge(): copies a[low..mid] and a[mid+1..high]  
// into temp arrays L and R, then repeatedly writes  
// back the smaller of L's front / R's front until  
// both temp arrays are fully consumed.
```

Merge Sort — Divide & Merge Example



Each level splits the array in half ($\log n$ levels); merging all levels back takes $O(n)$ work per level.

Merge Sort — Complexity Analysis

$O(n \log n)$

Best Case

$O(n \log n)$

Average Case

$O(n \log n)$

Worst Case — always splits evenly

- Space Complexity: $O(n)$ — needs auxiliary arrays for merging; this is its main drawback.
- Stable? Yes — the merge step always takes from the left array first when values are equal.
- Performance is consistent — no worst-case blow-up like Quick Sort, since the split is always balanced.
- Well suited for: sorting linked lists (no random access needed), external sorting (data too large for memory), and any application where stability and predictable performance matter.

RECAP

All Five Sorting Algorithms — At a Glance

Algorithm	Best	Average	Worst	Space	Stable	In-Place
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No

Selection, Insertion and Bubble Sort are simple but $O(n^2)$; Quick Sort and Merge Sort scale to large datasets at $O(n \log n)$.

Key Takeaways

- Searching finds an element's position; sorting arranges elements in order — sorted data unlocks much faster searching.
- Linear Search works on any data at $O(n)$; Binary Search needs sorted data but runs in $O(\log n)$; Interpolation Search can do even better, $O(\log \log n)$, on uniformly distributed sorted data.
- Selection, Insertion, and Bubble Sort are simple, in-place, $O(n^2)$ algorithms — good for small or nearly-sorted datasets, especially Insertion Sort.
- Quick Sort and Merge Sort use Divide and Conquer to reach $O(n \log n)$ — Quick Sort is faster in practice and in-place; Merge Sort is stable and has no worst-case blow-up, at the cost of $O(n)$ extra space.
- Choosing an algorithm always means trading off time complexity, space complexity, and stability against the shape of the real data.

Thank You

Questions & Discussion

Unit 4 — Searching and Sorting | Data Structures and Algorithms
Department of Information & Communication Technology, Parul University